

Tüm sabit değerleri ve değişiklikleri tek tek açıklıyorum:

1. Zaman_Carpani başlangıç değeri: 1.0

Neden 1.0? Sistem ilk açıldığında oynatma hızını henüz bilmez. 1.0 = "1x hız varsay" demektir. İlk paketler gelene kadar bu değerle çalışır. forceSync sonrasında da 1.0'a resetlenir çünkü yeni bir oturum başlıyor ve hız henüz ölçülmedi.

2. OPS hesabında dtLocal → dtPacket değişikliği

Eski: $\text{clampedDt} = \text{Math.max}(0.05, \text{Math.min}(\text{dtLocal}, 2.0))$ **Yeni:** $\text{clampedDtServer} = \text{Math.max}(0.05, \text{Math.min}(\text{dtPacket}, 2.0))$

Neden? Ortalama_Paket_Suresi sönümlleme rate'ini hesaplamak için kullanılıyor (normalRate = carpani / OPS). Fizik hızları simülasyon zamanında olduğu için OPS de simülasyon zamanında olmalı. 8x'te dtLocal=0.025s ama dtPacket=0.2s — eski kodda OPS=0.025s → rate=120 (aşırı agresif), yeni kodda OPS=0.2s → rate=15 (tutarlı).

Sınırlar (0.05 - 2.0): Önceki kodla aynı. 50ms'den kısa paket (saniyede 20+ paket) veya 2 saniyeden uzun boşluğu kırpar.

3. Zaman Çarpanı hesabı sabit değerleri

typescript

```
if (dtLocal > 0.005) { // 5ms

    const rawRatio = dtPacket / dtLocal;

    const clampedRatio = Math.max(0.1, Math.min(rawRatio, 32.0));

    if (this.Paket_Sayisi <= 2) {

        this.Zaman_Carpani = clampedRatio; // direkt set

    } else {

        this.Zaman_Carpani = this.Zaman_Carpani * 0.8 + clampedRatio * 0.2; // EMA

    }

}
```

- **0.005 (5ms):** dtLocal sıfıra çok yakınsa bölme patlar. 5ms = "paketler arasında en az 5ms geçmeli" anlamında güvenlik.
- **0.1 - 32.0 clamp aralığı:** 0.1x'ten yavaş veya 32x'ten hızlı oynatma desteklemiyoruz. Saçma oranların sistemi bozmasını engeller.
- **Paket_Sayisi <= 2 → direkt set:** Çarpan varsayılan 1.0'dan başlıyor. 8x'te ilk EMA: $1.0 * 0.8 + 8.0 * 0.2 = 2.4$ — yanlış! 8.0'a ulaşması ~15 paket sürer. İlk 2 pakette direkt clampedRatio atayarak anında doğru değere ulaşıyoruz.

- **0.8/0.2 EMA oranı:** Ağdaki paket zamanlama jitter'ını yumuşatır. %80 eski değeri koru, %20 yeniyi güven. Yaklaşık 5 pakette yeni değere yakınsar.

4. simDt ve realDt ayrımı

typescript

```
let realDt = (localNow - this.Son_Paket_Yerel_Zamanı) / 1000.0;
```

```
const simDt = realDt * this.Zaman_Carpanı;
```

Kullanım alanları:

Değişken	Kullanıldığı yer	Neden?
simDt	Ekstrapolasyon formülleri (CTRV, düz uçuş, dikey tahmin)	Fizik hızları sim zamanında → dt de sim zamanında olmalı
simDt	Timeout kontrolü (simDt > 3s)	"3 saniyelik simülasyon ilerlemesi" = veri eskimişliği
realDt	Sönümlleme ($\exp(-\text{rate} \times \text{realDt})$)	Hata eritme görsel efekt → izleyicinin gerçek zaman algısıyla uyumlu

5. gorsel_hiz — maxRate'te Yatay_Hiz → Yatay_Hiz × Zaman_Carpanı

typescript

```
const gorsel_hiz = this.Yatay_Hiz * this.Zaman_Carpanı;
```

```
const maxRate = gorsel_hiz / Math.max(pozisyon_hatasi_buyukluk, 0.01);
```

Neden? Çözüm A'nın garantisi: $\text{gorsel_hiz} - |\text{hata}| \times \text{rate} \geq 0$ (geri çekilme yok).

Sönümlleme realDt kullanıyor. Ekrandaki görsel hız ise $\text{sim_hiz} \times \text{oynatma_carpanı}$:

- 8x'te: $100 \text{ m/s} \times 8 = 800 \text{ m/s}$ görsel → maxRate yüksek → sönümlleme agresif olabilir (hata zaten küçük)
- 0.5x'te: $100 \text{ m/s} \times 0.5 = 50 \text{ m/s}$ görsel → maxRate düşer → yavaş uçan uçak gibi davranır

Aynı mantık yönelimde de uygulandı: $\text{gorsel_acisal_hiz} = \text{acisal_hiz} * \text{this.Zaman_Carpanı}$.

6. Dokunulmayan docstring (satır 402-413)

Dosyanın üst kısmındaki Guncel_Konumu_Getir docstring'i hâlâ "TEK SAAT MİMARİSİ" diyor — bu artık güncel değil. İstersen bunu da "İKİ ZAMANLI dt MİMARİSİ" olarak güncelleyebilirim.

Kısacası: **Yeni sabit değeri sadece Zaman_Carpanı = 1.0**. Diğer tüm sabitler (0.005, 0.1, 32.0, 0.8/0.2) ya mevcut kalıpların kopyası ya da matematiksel güvenlik sınırları. Temel mimari değişiklik dtSincePacket → simDt + realDt ayrımı.